

Ensemble Learning

Van-Thinh Vo^{1,2},

vovanthinhv225@gmail.com

¹Machine Learning & IoT Lab (ML IoT)

²Ho Chi Minh City University of Technology (HCMUT)

1. Motivation and Overviews
2. Bagging
3. Random Forest
4. Boosting
5. Stacking and Voting
6. Comparison and Practical Tips

Contact Information

Address: 403.1 BK.B6, HCMUT Campus 2

Email:

nkloi@hcmut.edu.vn

mlandiotlab@gmail.com

Website: mliotlab.com

Facebook: facebook.com/hcmut.ml.iot.lab

YouTube: youtube.com/@mliotlab

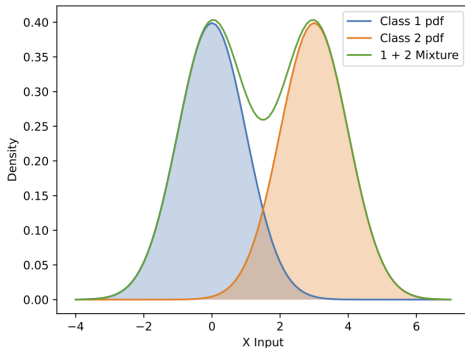
This slide was built upon Ensemble Learning Slide, Thanh-Sach LE, HCMUT - Email: Itsach@hcmut.edu.vn.

Motivation and Overview



Analogy

- ▶ A model is a combination of multi per-class probability distributions.

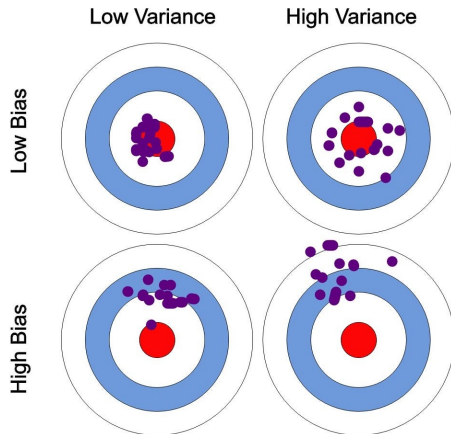


- ▶ **Single model:** Limited capacity, sensitive to the particular training sample.
- ▶ **Problem:** May suffer from high variance, high bias, or both.
- ▶ **Key idea:** Combine multiple models (learners) to obtain a more robust predictor.
- ▶ **Analogy:** "Wisdom of the crowd" - averaging many opinions can be more accurate than any individual opinion.

- ▶ **Bias:** Error due to wrong assumptions about the true function (underfitting).
- ▶ **Variance:** Error due to sensitivity to fluctuations in the training data (overfitting).
- ▶ **Ensemble goal:**
 - ▶ Reduce variance by averaging predictions of multiple models.
 - ▶ Sometimes also reduce bias by iteratively adding stronger models.
- ▶ Many ensemble methods exploit the fact that:

$$\text{Var} \left(\frac{1}{M} \sum_{m=1}^M \hat{y}^{(m)} \right) \approx \frac{1}{M^2} \sum_{m=1}^M \text{Var} \left(\hat{y}^{(m)} \right) \quad (1)$$

when models are weakly correlated.



Bagging

- ▶ **Goal:** Reduce variance of an unstable base learner (e.g., decision trees).
- ▶ **Bootstrap sample:** From training set of size n , sample n points with replacement.
- ▶ **Procedure:**
 - ▷ Train many models on different bootstrap samples.
 - ▷ Aggregate their predictions by averaging (regression) or voting (classification).
- ▶ **Effect:** Randomness in the training data for each model decorrelates errors, leading to variance reduction.

- ▶ Use BaggingClassifier with a decision tree as base learner.
- ▶ Train on a toy dataset and compare single tree vs bagging.

```
1 from sklearn.datasets import make_moons
2 from sklearn.model_selection import train_test_split
3 from sklearn.tree import DecisionTreeClassifier
4 from sklearn.metrics import accuracy_score
5
6 X, y = make_moons(n_samples=500, noise=0.3, random_state=0)
7 X_train, X_test, y_train, y_test = train_test_split(
8     X, y, test_size=0.3, random_state=0
9 )
10
11 # Single decision tree (high variance)
12 tree = DecisionTreeClassifier(max_depth=None, random_state=0)
13 tree.fit(X_train, y_train)
14 y_pred_tree = tree.predict(X_test)
```

- ▶ Add bagging on top of decision trees.
- ▶ Compare accuracies on the test set.

```
1 from sklearn.ensemble import BaggingClassifier
2
3 bagging = BaggingClassifier(
4     estimator=DecisionTreeClassifier(),
5     n_estimators=100,
6     bootstrap=True,
7     n_jobs=-1,
8     random_state=0
9 )
10 bagging.fit(X_train, y_train)
11 y_pred_bag = bagging.predict(X_test)
12
13 print("Tree accuracy: ", accuracy_score(y_test, y_pred_tree))
14 print("Bagging accuracy: ", accuracy_score(y_test, y_pred_bag))
```

Algorithm Bagging

Require: Training set $D = \{(x_i, y_i)\}_{i=1}^n$, base learner $\mathcal{L}$, number of models M

Ensure: Predictions $\hat{y}(x)$ for new input x

- 1: **for** $m = 1$ **to** M **do**
 - 2: Sample bootstrap dataset D_m by drawing n samples with replacement from D
 - 3: Train base model $h_m = \mathcal{L}(D_m)$
 - 4: **end for**
 - 5: **for** a new input x **do**
 - 6: **Regression:** $\hat{y}(x) = \frac{1}{M} \sum_{m=1}^M h_m(x)$
 - 7: **Classification:** $\hat{y}(x) = \text{majority_vote}(h_1(x), \dots, h_M(x))$
 - 8: **end for**
-

- ▶ **Best for:** High-variance, low-bias models (e.g., full decision trees).
- ▶ **Pros:**
 - ▷ Simple to implement.
 - ▷ Parallelizable: all models can be trained independently.
 - ▷ Strong reduction in variance.
- ▶ **Cons:**
 - ▷ Large storage cost (many models).
 - ▷ Slower prediction than a single model.
 - ▷ Harder to interpret than one tree.

Random Forest

- ▶ **Base learner:** Decision tree.
- ▶ **Two sources of randomness:**
 - ▶ Bootstrap sampling of the training data for each tree.
 - ▶ Random subset of features considered at each split.
- ▶ **Motivation:** Reduce correlation between trees to obtain stronger variance reduction.
- ▶ **Prediction:** Average (regression) or majority vote (classification) over all trees.

Algorithm Random Forest Training

Require: Training set $D = \{(x_i, y_i)\}_{i=1}^n$ with d features, number of trees M

```
1: for  $m = 1$  to  $M$  do
2:   Sample bootstrap dataset  $D_m$  by drawing  $n$  samples with replacement from  $D$ 
3:   Build decision tree  $T_m$  on  $D_m$ :
4:   while stopping criterion not met do
5:     Randomly select subset of features  $\mathcal{F}_{\text{sub}}$ 
6:     for each feature in  $\mathcal{F}_{\text{sub}}$  do
7:       Find best split point (e.g., maximize information gain)
8:     end for
9:     Choose best split among  $\mathcal{F}_{\text{sub}}$  (not all features) and split the dataset.
10:  end while
11: end for
12: Return ensemble  $\{T_1, T_2, \dots, T_M\}$ 
```

- ▶ **At each split in a tree:**
 - ▷ Randomly select a subset of features $\mathcal{F}_{\text{sub}}$.
 - ▷ Find the best split using only $\mathcal{F}_{\text{sub}}$.
- ▶ **Typical choice:**
 - ▷ Classification: $|\mathcal{F}_{\text{sub}}| = \sqrt{d}$.
 - ▷ Regression: $|\mathcal{F}_{\text{sub}}| = \frac{d}{3}$.
- ▶ **Key hyperparameters:**
 - ▷ Number of trees M .
 - ▷ Maximum depth of each tree.
 - ▷ Minimum samples per leaf / split.



```
1  from sklearn.tree import DecisionTreeClassifier
2  from sklearn.ensemble import RandomForestClassifier
3
4  # Initialize Random Forest
5  rf = RandomForestClassifier(n_estimators=100, random_state=0)
6  rf.fit(X_train, y_train)
7
8  # Single decision tree (high variance)
9  tree = DecisionTreeClassifier(max_depth=None, random_state=0)
10 tree.fit(X_train, y_train)
11 y_pred_tree = tree.predict(X_test)
12
13 # Evaluate and compare
14 y_pred_rf = rf.predict(X_test)
15 rf_acc = accuracy_score(y_test, y_pred_rf)
16 tree_acc = accuracy_score(y_test, y_pred_tree)
17
18 print(f"Decision Tree Accuracy: {tree_acc:.4f}")
19 print(f"Random Forest Accuracy: {rf_acc:.4f}")
```

Pros:

- ▶ Strong performance on many tabular datasets.
- ▶ Handles mixed feature types and non-linear interactions.
- ▶ Built-in estimate of feature importance.

Cons:

- ▶ Less interpretable than a single decision tree.
- ▶ Large memory footprint and slower inference.
- ▶ May underperform gradient boosting on highly structured/tabular data.

Boosting

- ▶ Sequential ensemble: Train models one after another.
- ▶ Each new model focuses on **examples that previous models predicted poorly**.
- ▶ Base learners are typically **weak** (e.g., shallow trees).
- ▶ **Contrast with bagging:**
 - ▷ Bagging: models are trained independently and can be parallelized.
 - ▷ Boosting: models are dependent and training is sequential.

1. Consider binary classification $y_i \in \{-1, +1\}$.
2. Maintain a **weight** w_i for each training example.
3. Train a weak learner on the weighted dataset.
4. Increase the weights of misclassified examples and decrease weights of correctly classified ones.
5. Final classifier is a **weighted majority vote** of weak learners.

Algorithm AdaBoost (Binary Classification)

Require: Training set $\{(x_i, y_i)\}_{i=1}^n$ with $y_i \in \{-1, +1\}$, number of iterations T , weak learner $\mathcal{L}$

Ensure: Final classifier $H(x)$

- 1: Initialize weights $w_i^{(1)} = \frac{1}{n}$ for all $i = 1, \dots, n$
 - 2: **for** $t = 1$ **to** T **do**
 - 3: Train weak learner h_t using weights $w_i^{(t)}$
 - 4: Compute weighted error $\varepsilon_t = \frac{\sum_{i=1}^n w_i^{(t)} \mathbb{I}[h_t(x_i) \neq y_i]}{\sum_{i=1}^n w_i^{(t)}}$
 - 5: Compute learner weight $\alpha_t = \frac{1}{2} \ln \frac{1 - \varepsilon_t}{\varepsilon_t}$
 - 6: Update weights: $w_i^{(t+1)} = w_i^{(t)} \exp(-\alpha_t y_i h_t(x_i))$
 - 7: Normalize $w_i^{(t+1)}$ so that $\sum_i w_i^{(t+1)} = 1$
 - 8: **end for**
 - 9: **Return** $H(x) = \text{sign}\left(\sum_{t=1}^T \alpha_t h_t(x)\right)$
-

- ▶ At iteration t , we maintain a weight $w_i^{(t)}$ for each training example (x_i, y_i) .
- ▶ The weights are normalized so that:

$$\sum_{i=1}^n w_i^{(t)} = 1,$$

so they define a probability distribution over training examples.

- ▶ **Interpretation:**
 - ▶ Large $w_i^{(t)}$: example i is “important” and should be fitted well.
 - ▶ Small $w_i^{(t)}$: example i is currently less important.
- ▶ Initial step: all examples are equally important,

$$w_i^{(1)} = \frac{1}{n}.$$

- ▶ Train a weak learner h_t using the current weights $w_i^{(t)}$.
- ▶ The learner sees a **weighted** training set: misclassified examples with large weights hurt more.
- ▶ Weighted error:

$$\varepsilon_t = \frac{\sum_{i=1}^n w_i^{(t)} \mathbb{I}[h_t(x_i) \neq y_i]}{\sum_{i=1}^n w_i^{(t)}}.$$

- ▶ **Meaning:**
 - ▷ Numerator: total weight of misclassified examples.
 - ▷ Denominator: total weight of all examples (equal to 1 after normalization).
 - ▷ ε_t is the misclassification rate under the current weight distribution.

Weak Learner + better than random

- ▶ h_t is a weak learner, but it is better than random: $\varepsilon_t < 0.5$.
- ▶ If $\varepsilon_t < 0.5$: continue training h_t on the next iteration; otherwise stop.

- ▶ Compute

$$\alpha_t = \frac{1}{2} \ln \frac{1 - \varepsilon_t}{\varepsilon_t}.$$

- ▶ h_t is better than random: $\varepsilon_t < 0.5$:
 - ▶ Then $\frac{1 - \varepsilon_t}{\varepsilon_t} > 1$ and $\alpha_t > 0$.
 - ▶ Smaller ε_t (more accurate h_t) implies larger α_t .
- ▶ **Interpretation:**
 - ▶ α_t is the “vote weight” of h_t in the final classifier.
 - ▶ Stronger weak learners (lower error) receive higher influence.

► **Update:**

$$w_i^{(t+1)} = w_i^{(t)} \exp(-\alpha_t y_i h_t(x_i)) .$$

► Assume $y_i \in \{-1, +1\}$ and $h_t(x_i) \in \{-1, +1\}$.

► **If h_t predicts correctly:** $y_i h_t(x_i) = 1$

$$w_i^{(t+1)} = w_i^{(t)} e^{-\alpha_t} \quad \Rightarrow \quad \text{weight decreases.}$$

► **If h_t predicts incorrectly:** $y_i h_t(x_i) = -1$

$$w_i^{(t+1)} = w_i^{(t)} e^{+\alpha_t} \quad \Rightarrow \quad \text{weight increases.}$$

► Misclassified examples become more important in the next iteration.

- ▶ After updating, renormalize:

$$w_i^{(t+1)} \leftarrow \frac{w_i^{(t+1)}}{\sum_{j=1}^n w_j^{(t+1)}} \implies \sum_{i=1}^n w_i^{(t+1)} = 1.$$

- ▶ At the end, combine all weak learners:

$$H(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(x) \right).$$

- ▶ **View:** $H(x)$ is a **weighted majority vote**:
 - ▶ Each h_t casts a vote $h_t(x) \in \{-1, +1\}$.
 - ▶ Vote strength is α_t .
- ▶ Final decision uses the sign of the weighted sum of all votes.



- ▶ Use AdaBoostClassifier with shallow decision trees (stumps) as weak learners.
- ▶ Train on a toy dataset and compare a single tree vs AdaBoost.

```
1 from sklearn.tree import DecisionTreeClassifier
2
3 # Single shallow decision tree (weak learner)
4 tree = DecisionTreeClassifier(max_depth=1, random_state=0)
5 tree.fit(X_train, y_train)
6 y_pred_tree = tree.predict(X_test)
```



- ▶ Wrap the weak learner in AdaBoostClassifier.
- ▶ Compare test accuracy of the single tree vs AdaBoost.

```
1 from sklearn.ensemble import AdaBoostClassifier
2
3 ada = AdaBoostClassifier(
4     estimator=DecisionTreeClassifier(max_depth=1),
5     n_estimators=100,
6     learning_rate=0.5,
7     random_state=0
8 )
9 ada.fit(X_train, y_train)
10 y_pred_ada = ada.predict(X_test)
11
12 print("Tree accuracy: ", accuracy_score(y_test, y_pred_tree))
13 print("AdaBoost accuracy: ", accuracy_score(y_test, y_pred_ada))
```

- ▶ General framework for regression and classification.
- ▶ View boosting as **gradient descent** in function space.
- ▶ At iteration t :
 - ▶ Compute pseudo-residuals (negative gradient of loss w.r.t. predictions).
 - ▶ Fit a new tree h_t to these residuals (**note**: h_t is a regression task, **even for classification problems**).
 - ▶ Update the model: $F_t(x) = F_{t-1}(x) + \eta h_t(x)$
 - ▶ η is the **learning rate** (step size).
- ▶ Implementations: XGBoost, LightGBM, CatBoost, etc.

- ▶ Training data: $D = \{(x_i, y_i)\}_{i=1}^n$.

- ▶ Model is an additive function:

$$F_T(x) = \sum_{t=1}^T \eta_t h_t(x),$$

where each h_t is a weak learner (e.g., regression tree).

- ▶ We minimize a differentiable empirical loss:

$$\mathcal{L}(F) = \sum_{i=1}^n \ell(y_i, F(x_i)).$$

- ▶ **Idea:** At step t , move F_{t-1} in the direction of the **negative gradient** of $\mathcal{L}$ with respect to F .

- ▶ Loss for regression:

$$\ell(y_i, F(x_i)) = \frac{1}{2}(y_i - F(x_i))^2.$$

- ▶ Gradient with respect to prediction $F(x_i)$:

$$\frac{\partial \ell}{\partial F(x_i)} = F(x_i) - y_i.$$

- ▶ Negative gradient (pseudo-residual):

$$r_i^{(t)} = -\left. \frac{\partial \ell}{\partial F(x_i)} \right|_{F=F_{t-1}} = y_i - F_{t-1}(x_i).$$

- ▶ **Algorithm step:**

- ▶ Compute residuals $r_i^{(t)} = y_i - F_{t-1}(x_i)$.
- ▶ Fit a regression tree h_t to data $\{(x_i, r_i^{(t)})\}$.
- ▶ Update: $F_t(x) = F_{t-1}(x) + \eta h_t(x)$.

- ▶ Assume $y_i \in \{-1, +1\}$ and model $F(x)$ outputs a real score.
- ▶ Use logistic (binomial deviance) loss:

$$\ell(y_i, F(x_i)) = \log(1 + \exp(-2y_i F(x_i))).$$

- ▶ Gradient with respect to $F(x_i)$:

$$\frac{\partial \ell}{\partial F(x_i)} = -\frac{2y_i}{1 + \exp(2y_i F(x_i))}.$$

- ▶ Negative gradient (pseudo-residual):

$$r_i^{(t)} = \frac{2y_i}{1 + \exp(2y_i F_{t-1}(x_i))}.$$

- ▶ **Algorithm step:**

- ▶ Compute $r_i^{(t)}$ for all training points.
- ▶ Fit a regression tree h_t to $(x_i, r_i^{(t)})$.
- ▶ Update: $F_t(x) = F_{t-1}(x) + \eta h_t(x)$.



- Initialize and training GradientBoostingClassifier on simple dataset.

```
1 from sklearn.ensemble import GradientBoostingClassifier
2
3 # Inititalize Gradient Boosting
4 gbrt = GradientBoostingClassifier(
5     n_estimators=100, learning_rate=0.1,
6     max_depth=3, random_state=0
7 )
8 gbrt.fit(X_train, y_train)
9
10 # Evaluate
11 y_pred_gb = gbrt.predict(X_test)
12 gb_acc = accuracy_score(y_test, y_pred_gb)
13
14 print(f"Gradient Boosting Accuracy: {gb_acc:.4f}")
```

► **Pros:**

- ▶ Often achieves state-of-the-art performance on tabular data.
- ▶ Can reduce both bias and variance.
- ▶ Flexible choice of loss functions.

► **Cons:**

- ▶ More sensitive to noise and outliers than bagging.
- ▶ Requires careful tuning (number of trees, depth, learning rate).
- ▶ Training is sequential and less parallelizable.

XGBoost: A Scalable Tree Boosting System

- ▶ **XGBoost** = eXtreme Gradient **B**oosting
- ▶ **Regularization**: penalize more on trees which are too complex/deep, avoid overfitting.
- ▶ **Sparsity**: handle missing attribute efficiently.
- ▶ **Speed**: faster 10× than other conventional boosting algorithms.
- ▶ **Engineering Optimization**: Column Block, Cach-aware access, Out-of-core.

- ▶ **Exact Greedy:** theoretically, calculate information gain on all values of a feature to find the best. It only is suitable for small and medium dataset.
- ▶ **Approximate:** it propose some *candidate* values based on quantiles. There are two types:
 - ▶ **Global:** choose split points once for trees.
 - ▶ **Local:** choose again after split.

- ▶ **Compared to scikit-learn and R's gbm:** XGBoost is 10 to 40 times faster, while achieving similar or better accuracy.
- ▶ **Compared to pGBRT, Spark MLlib, and H2O:** XGBoost is 2 to 10 times faster. More importantly, it can process the entire dataset of 1.7 billion samples, whereas the other systems can only handle a small subset due to memory constraints.
- ▶ **Accuracy and practical impact:** XGBoost was the winning solution in 17 out of 29 Kaggle competitions in 2015. Furthermore, all top-10 teams in KDDCup 2015 used XGBoost.

Requirement!

- ▶ Firstly, install xgboost package via
`pip install xgboost`

```
1  from xgboost import XGBClassifier
2
3  # Initialize a XGBosst Classifier
4  xgb_clf = XGBClassifier(
5      n_estimators=100, learning_rate=0.1, max_depth=3,
6      random_state=0, use_label_encoder=False, eval_metric='logloss'
7  )
8  xgb_clf.fit(X_train, y_train)
9
10 # Evaluate
11 y_pred_xgb = xgb_clf.predict(X_test)
12 xgb_acc = accuracy_score(y_test, y_pred_xgb)
13 print(f"XGBoost Accuracy: {xgb_acc:.4f}")
```

- ▶ **LightGBM:** A modern, highly efficient variant that uses a leaf-wise tree growth strategy to achieve faster training speeds and lower memory usage compared to XGBoost.
- ▶ **CatBoost:** A modern boosting algorithm specifically optimized for categorical features. It uses ordered boosting to prevent prediction shift and often requires less feature pre-processing.

Stacking and Voting

- ▶ **Voting classifier:** Combine several classifiers by majority (hard) or weighted (soft) voting.
- ▶ **Averaging regressor:** Average predictions of several regressors.
- ▶ **Use case:** Combine strong but diverse models (e.g., SVM, random forest, neural network).
- ▶ Simple but often effective baseline ensemble.

- ▶ Combine different classifiers (logistic regression, decision tree, SVM).
- ▶ Use soft voting (average predicted probabilities).

```
1 from sklearn.datasets import make_classification
2 from sklearn.model_selection import train_test_split
3
4 X, y = make_classification(
5     n_samples=500, n_features=10,
6     n_informative=5, n_redundant=0,
7     random_state=0
8 )
9
10 X_train, X_test, y_train, y_test = train_test_split(
11     X, y, test_size=0.3, random_state=0
12 )
```

- ▶ Define the base classifiers.

```
1 from sklearn.linear_model import LogisticRegression
2 from sklearn.tree import DecisionTreeClassifier
3 from sklearn.svm import SVC
4
5 clf1 = LogisticRegression(max_iter=1000, random_state=0)
6 clf2 = DecisionTreeClassifier(max_depth=5, random_state=0)
7 clf3 = SVC(probability=True, random_state=0)
```

- ▶ Create a VotingClassifier with soft voting.
- ▶ Compare accuracy on the test set.

```
1 from sklearn.ensemble import VotingClassifier
2 from sklearn.metrics import accuracy_score
3
4 voting_clf = VotingClassifier(
5     estimators=[('lr', clf1), ('dt', clf2), ('svm', clf3)],
6     voting='soft'
7 )
8
9 voting_clf.fit(X_train, y_train)
10 y_pred = voting_clf.predict(X_test)
11
12 print("Voting accuracy:", accuracy_score(y_test, y_pred))
```

- ▶ **Level-0 models (base learners):** Train multiple different models on the same training data.
- ▶ **Level-1 model (meta-learner):** Takes as input the predictions of level-0 models and outputs the final prediction.
- ▶ **Training strategy:**
 - ▶ Use cross-validation to generate out-of-fold predictions from base learners.
 - ▶ Use these predictions (and possibly original features) to train the meta-learner.
- ▶ **Advantage:** Learns how to optimally combine heterogeneous models.

- ▶ Suppose we have M base learners (level-0):

$$h_1(x), h_2(x), \dots, h_M(x).$$

- ▶ For each input x , base learners produce predictions:

$$z(x) = (h_1(x), \dots, h_M(x)).$$

- ▶ The meta-learner (level-1) is a model g trained on $z(x)$ (and optionally original x):

$$\hat{y}(x) = g(z(x)) \quad \text{or} \quad \hat{y}(x) = g(x, z(x)).$$

- ▶ **Key idea:** Learn how to combine base predictions instead of using fixed rules (as in voting).

- ▶ Directly training g on predictions of h_m from the same data causes overfitting.
- ▶ Solution: use K -fold cross-validation to get **out-of-fold** predictions.
- ▶ For each base learner h_m :
 - ▶ Split training data into K folds.
 - ▶ For each fold k : train h_m on other $K - 1$ folds, predict on fold k .
 - ▶ Concatenate predictions over all folds to get out-of-fold vector

$$z_{i,m} = h_m^{(k)}(x_i),$$

where x_i belongs to fold k .

- ▶ After obtaining out-of-fold predictions $\{z_{i,m}\}$ for all base learners $m = 1, \dots, M$:
 - ▶ Construct a new feature vector for each training example:

$$z_i = (z_{i,1}, \dots, z_{i,M}).$$

- ▶ The corresponding target remains y_i .
- ▶ **Meta-training dataset:**
$$\{(z_i, y_i)\}_{i=1}^n.$$
- ▶ Choose any suitable supervised learner as meta-model g :
 - ▶ Classification: logistic regression, linear SVM, random forest, etc.
 - ▶ Regression: linear regression, ridge, random forest regressor, etc.
- ▶ Train g on (z_i, y_i) exactly as in a standard supervised learning problem.

Voting

- ▶ Fixed rule (majority vote or average)
- ▶ No additional model trained

Stacking

- ▶ Learn meta-model g
- ▶ Capture complex relationships

Trade-offs:

- ▶ Stacking: more flexible, accurate, but complex and slower
- ▶ Voting: simpler, easier to interpret

- Setup: Import libraries and create toy dataset.

```
1 from sklearn.datasets import make_classification
2 from sklearn.model_selection import train_test_split
3 from sklearn.linear_model import LogisticRegression
4 from sklearn.ensemble import RandomForestClassifier
5 from sklearn.svm import SVC
6 from sklearn.metrics import accuracy_score
7
8 X, y = make_classification(
9     n_samples=500, n_features=10,
10    n_informative=5, n_redundant=0,
11    random_state=0
12 )
13
14 X_train, X_test, y_train, y_test = train_test_split(
15     X, y, test_size=0.3, random_state=0
16 )
```

- ▶ Create base learners (level-0) and meta-learner (level-1).
- ▶ Then create StackingClassifier.

```
1 # Base learners (level-0)
2 clf_lr = LogisticRegression(max_iter=1000, random_state=0)
3 clf_rf = RandomForestClassifier(n_estimators=100, random_state=0)
4 clf_svc = SVC(probability=True, random_state=0)
5
6 # Meta-learner (level-1)
7 meta_lr = LogisticRegression(max_iter=1000, random_state=0)
8
9 # Create StackingClassifier
10 from sklearn.ensemble import StackingClassifier
11
12 stack_clf = StackingClassifier(
13     estimators=[('lr', clf_lr), ('rf', clf_rf), ('svc', clf_svc)],
14     final_estimator=meta_lr
15 )
```

- ▶ Train models and compare performance.

```
1 # Fit baseline and stacking models
2 clf_lr.fit(X_train, y_train)
3 stack_clf.fit(X_train, y_train)
4
5 # Evaluate and compare
6 print("LogisticRegression_acc:",
7       accuracy_score(y_test, clf_lr.predict(X_test)))
8 print("Stacking_acc:",
9       accuracy_score(y_test, stack_clf.predict(X_test)))
```

Expected Output

- ▶ Stacking typically achieves higher accuracy than individual base models.
- ▶ The meta-learner learns to combine predictions optimally.

Comparison and Practical Tips

▶ **Bagging:**

- ▶ Reduces variance.
- ▶ Works best with high-variance models (e.g., deep trees).

▶ **Boosting:**

- ▶ Sequentially focuses on hard examples.
- ▶ Can reduce both bias and variance, but may overfit noisy data.

▶ **Stacking:**

- ▶ Combines heterogeneous models via a meta-learner.
- ▶ More flexible but also more complex to implement and tune.

▶ **Use them when:**

- ▶ Single models underperform on complex datasets.
- ▶ You can afford extra training and prediction time.

▶ **Be careful about:**

- ▶ Overfitting in boosting with too many iterations or deep trees.
- ▶ Interpretability vs accuracy trade-off.
- ▶ Computational cost on very large datasets.

▶ **Practical tip:** Random Forest and Gradient Boosting are strong default choices for tabular data.

- [1] Tianqi Chen, Carlos Guestrin. (2016). *XGBoost: A Scalable Tree Boosting System*. In *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*, pp. 785-794. DOI: [10.1145/2939672.2939785](https://doi.org/10.1145/2939672.2939785)
- [2] Ke, Guolin, et al (2017). *Lightgbm: A highly efficient gradient boosting decision tree*. In *Advances in neural information processing systems*, Vol(30).
- [3] Prokhorenkova, Liudmila, et al. (2018). *CatBoost: unbiased boosting with categorical features*. In *Advances in neural information processing systems*, Vol(31), pp. 6639 - 6649. DOI: [10.48550/arXiv.1706.09516](https://doi.org/10.48550/arXiv.1706.09516)

Thank You

Acknowledgements

The authors gratefully acknowledge MLIoT Lab Summer Courses TA for their support and guidance.